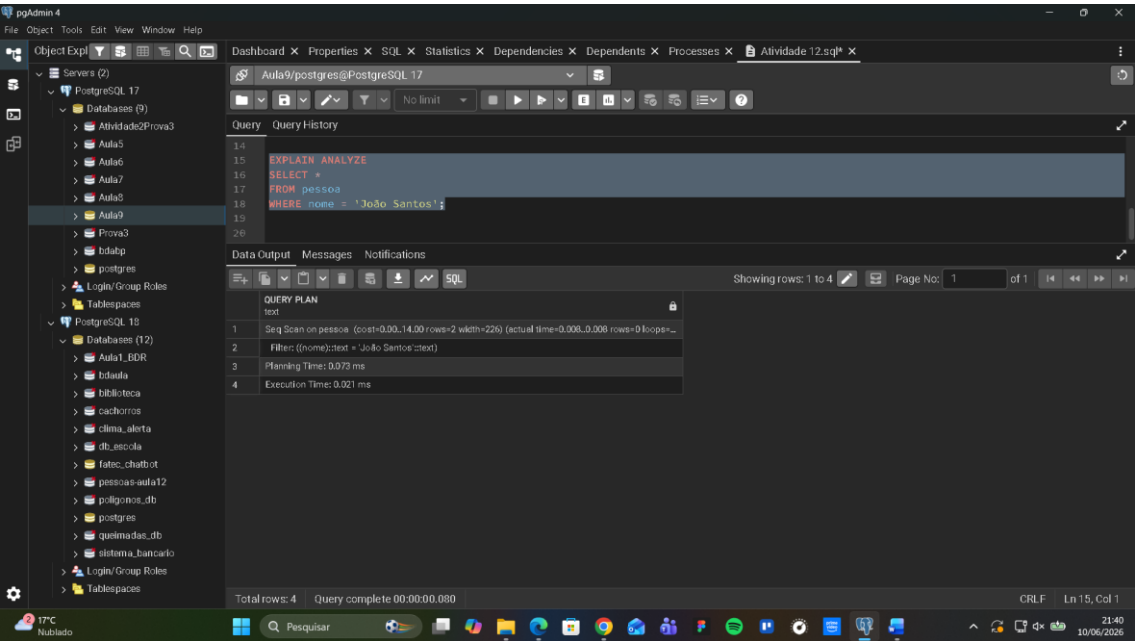
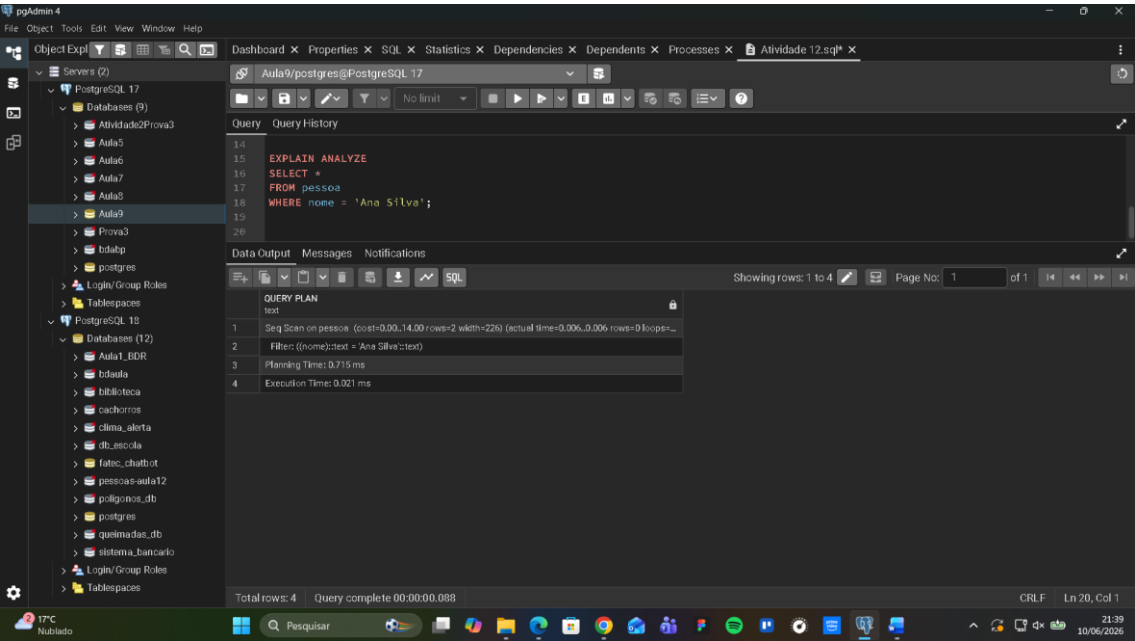


Atividade aula 12

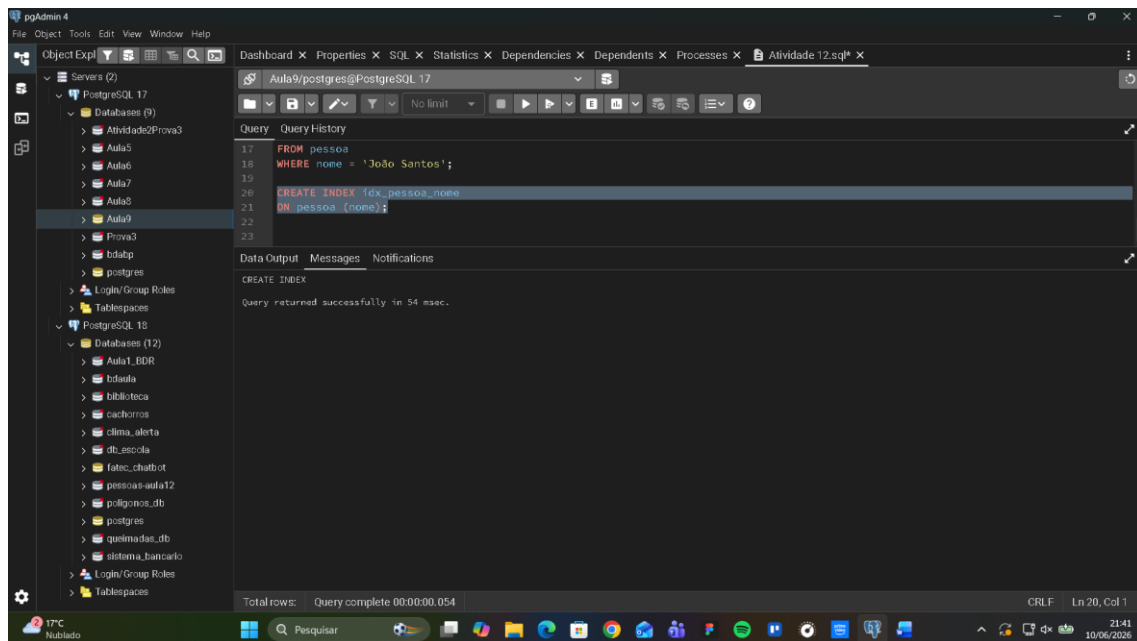
Luka Gomes

Exercício 1-

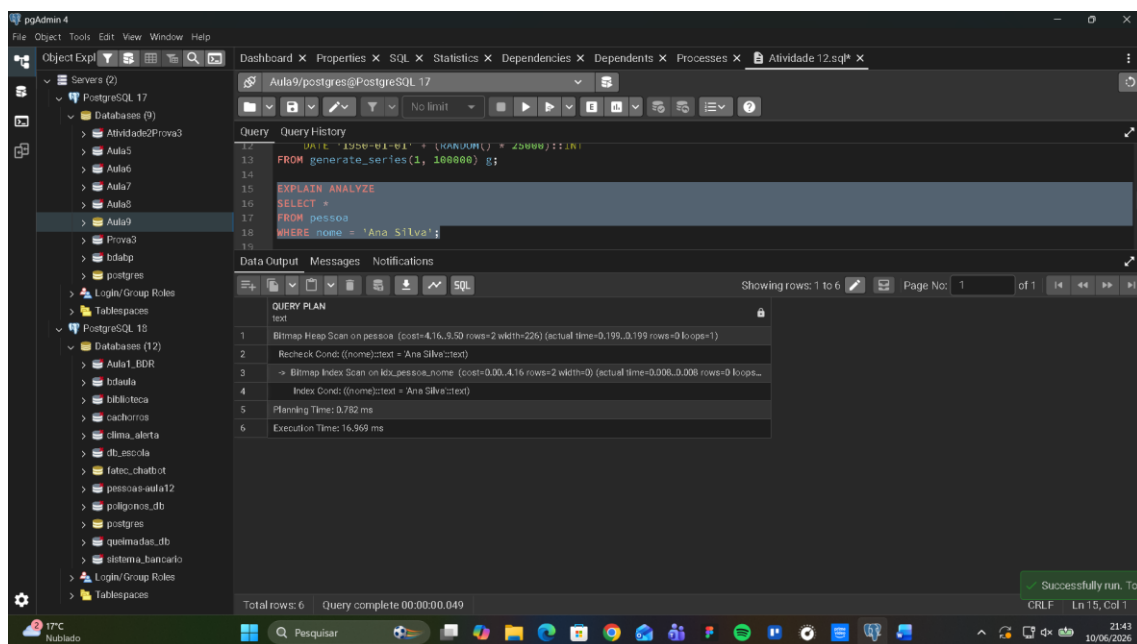
Parte A – Consulta sem índice explícito

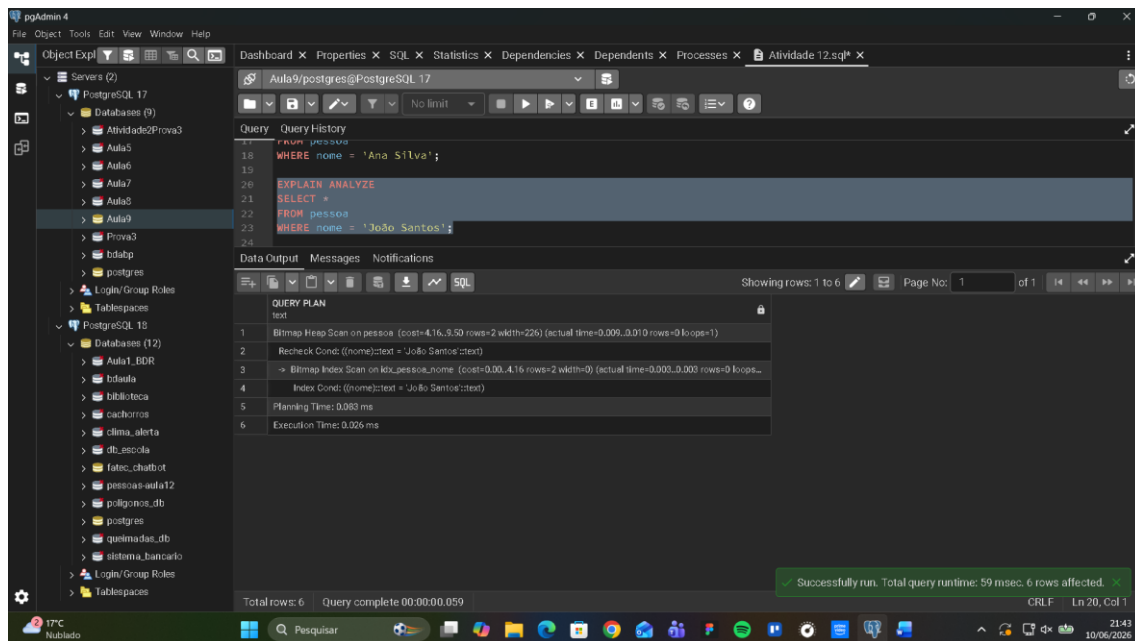


Parte B – Criação do índice



Parte C – Consulta com índice





Parte D – Responda, de forma objetiva:

- Qual plano foi utilizado antes do índice?

Seq scan

- Qual plano foi utilizado após o índice?

Bitmap Heap Scan

- Houve redução no tempo de execução?

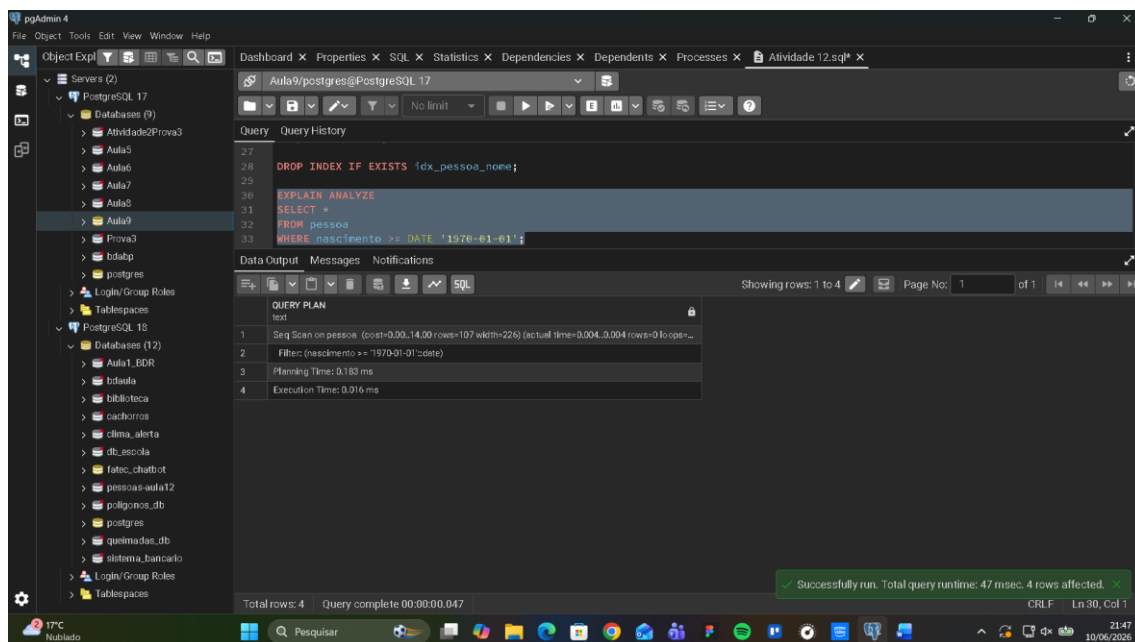
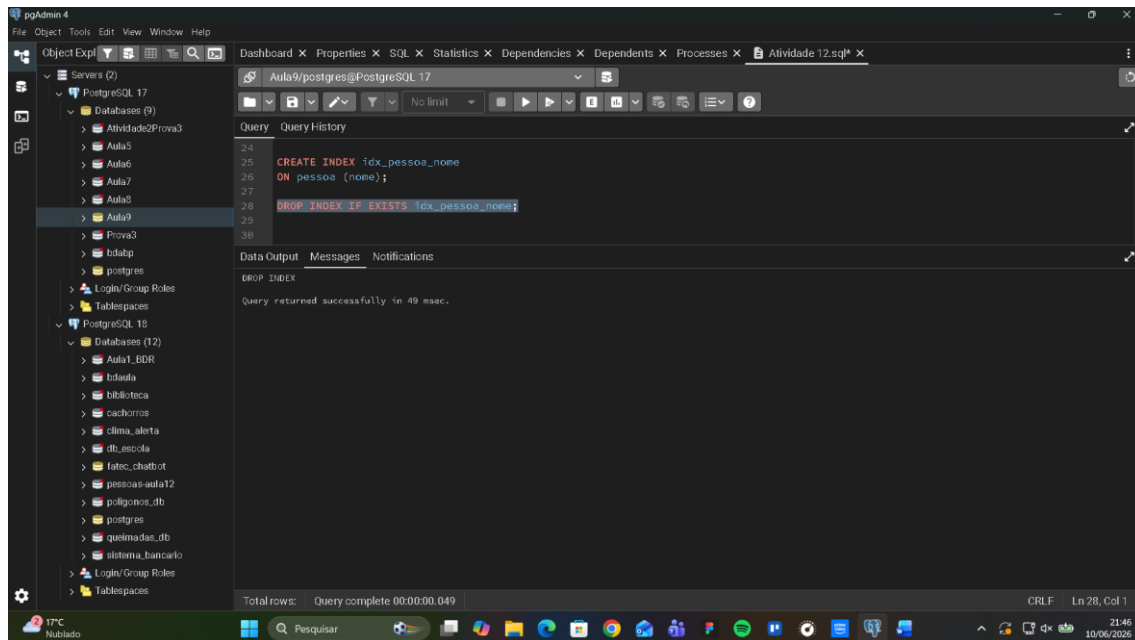
Sim

- O índice foi utilizado em ambos os nomes testados? Justifique com base na saída do plano.

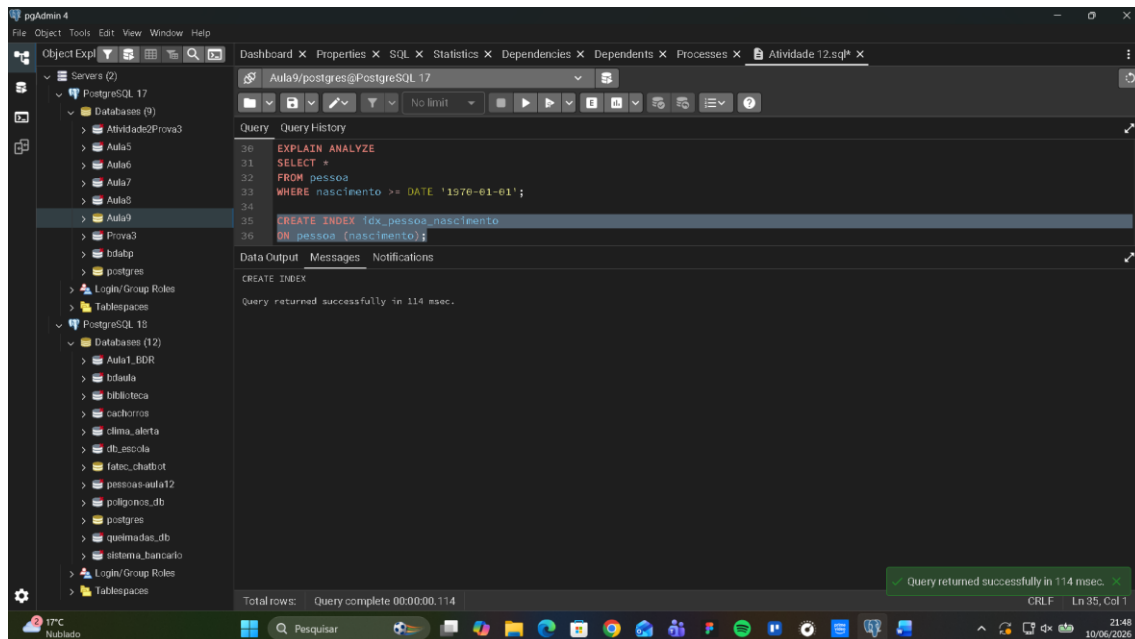
Sim, é possível verificar nas linhas 4 e 5 que o índice idx_pessoa_nome foi acionado, confirmando que o otimizador utilizou o índice para ambas as consultas.

Exercício 2

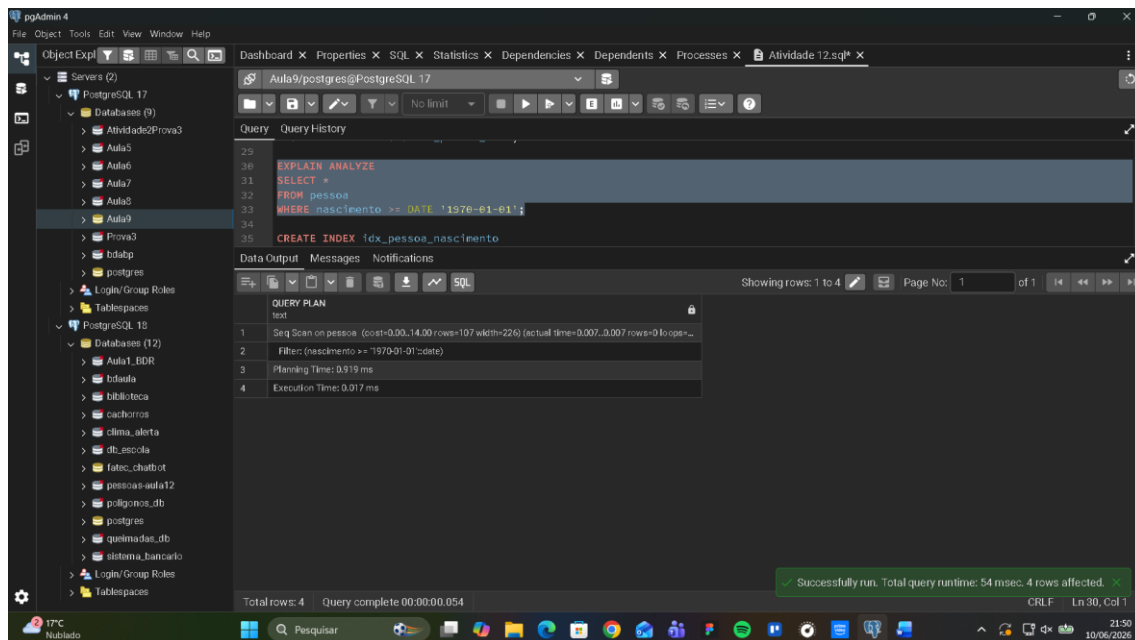
Parte A – Consulta por intervalo sem índice explícito



Parte B – Criação de índice na coluna nascimento



Parte C – Consulta com índice criado



Parte D – Responda, de forma objetiva

- O índice criado foi utilizado pelo PostgreSQL?

Não

- O plano de execução mudou após a criação do índice?

Não

- Houve melhora significativa no tempo de execução?

Não

- Por que o otimizador pode optar por não utilizar um índice, mesmo quando ele existe?
Se a consulta retorna muitos registros, o índice não compensa e o banco prefere o Seq Scan.

Exercício 3

Parte A – Consulta sem índice composto

The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree is expanded to show 'PostgreSQL 17' and its databases. The 'Query' tab is active, displaying the following SQL code:

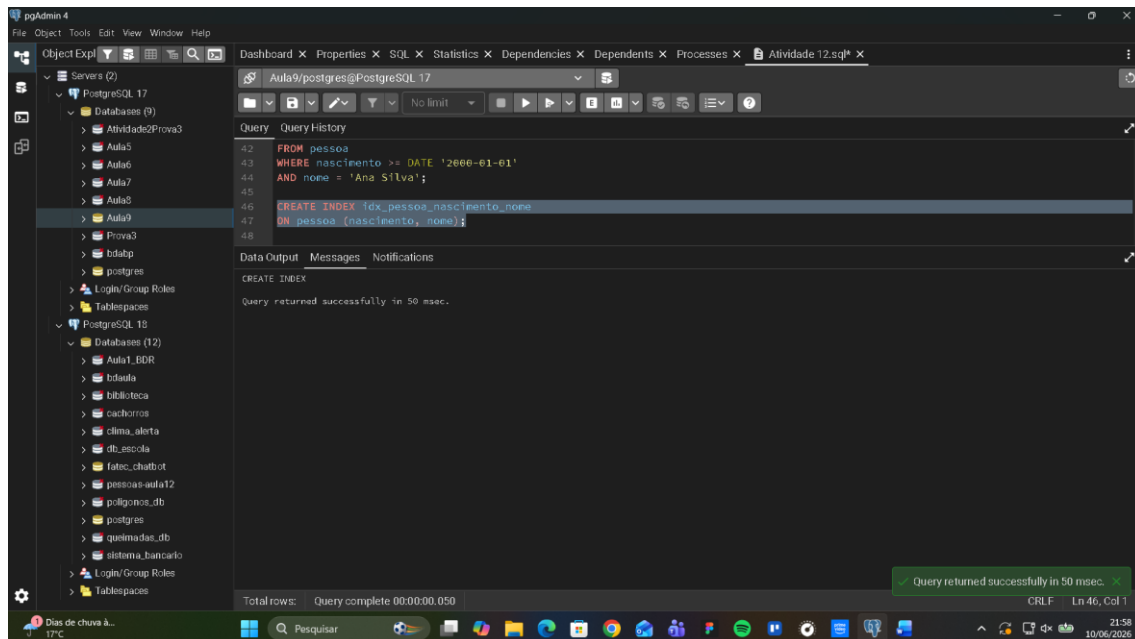
```
38 DROP INDEX IF EXISTS idx_pessoa_nascimento;
39
40 EXPLAIN ANALYZE
41 SELECT *
42 FROM pessoa
43 WHERE nascimento >= DATE '2000-01-01'
44 AND nome = 'Ana Silva';
```

Below the query, the 'Data Output' tab shows the 'QUERY PLAN' results:

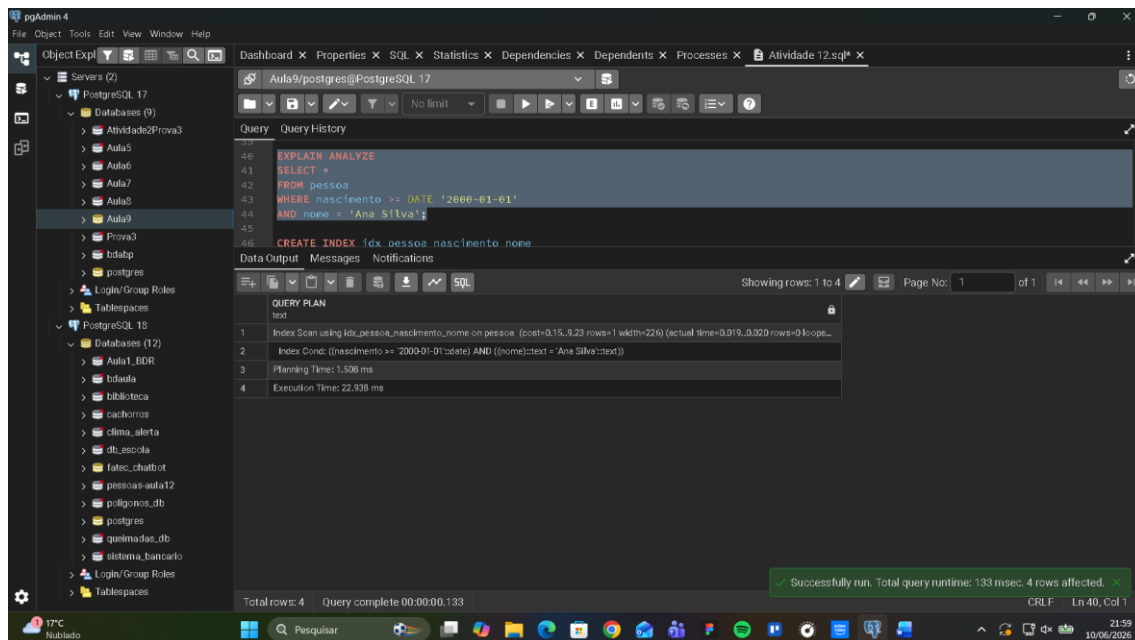
Step	Plan
1	Seq Scan on pessoa (cost=0.00..14.80 rows=1 width=226) (actual time=0.007..0.007 rows=0 loops=0)
2	Filter: ((nascimento >= '2000-01-01'::date) AND ((nome)::text = 'Ana Silva'::text))
3	Planning Time: 0.234 ms
4	Execution Time: 0.002 ms

At the bottom, a status bar indicates 'Total rows: 4' and 'Query complete 00:00:00.053'. A green message box at the bottom right states 'Successfully run. Total query runtime: 53 msec. 4 rows affected.'

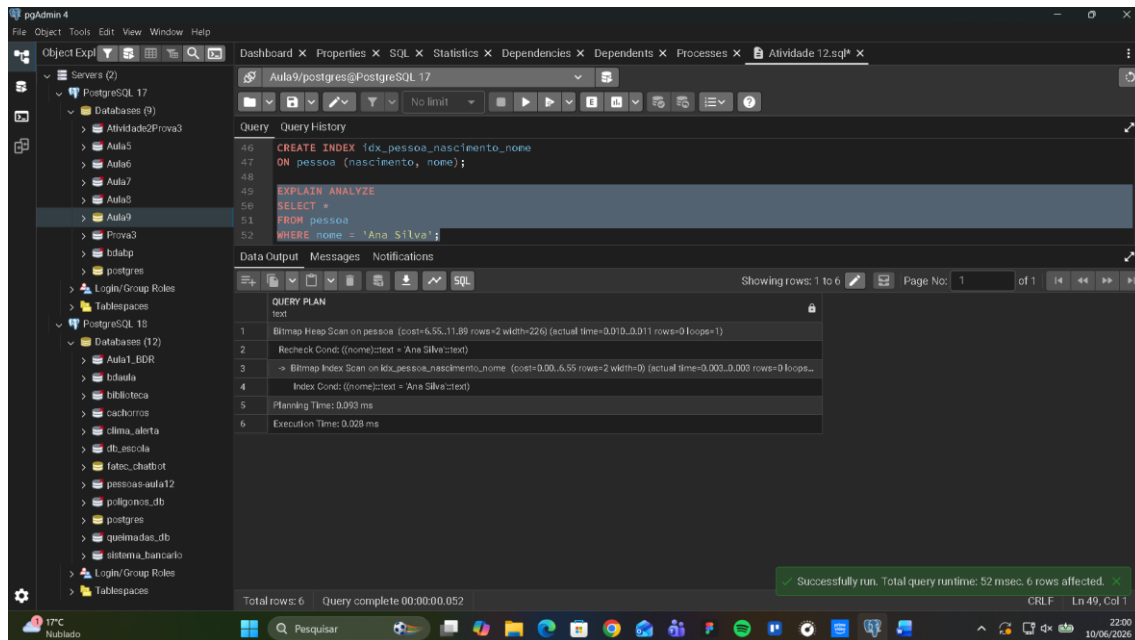
Parte B – Criação do índice composto



Parte C – Consulta com índice composto



Parte D – Teste da ordem das colunas



Parte E – Responda, de forma objetiva

- Qual plano foi utilizado antes da criação do índice composto?

Seq scan

- Qual plano foi utilizado após a criação do índice composto?

Index scan

- O índice composto foi utilizado na consulta que filtra apenas por nome?

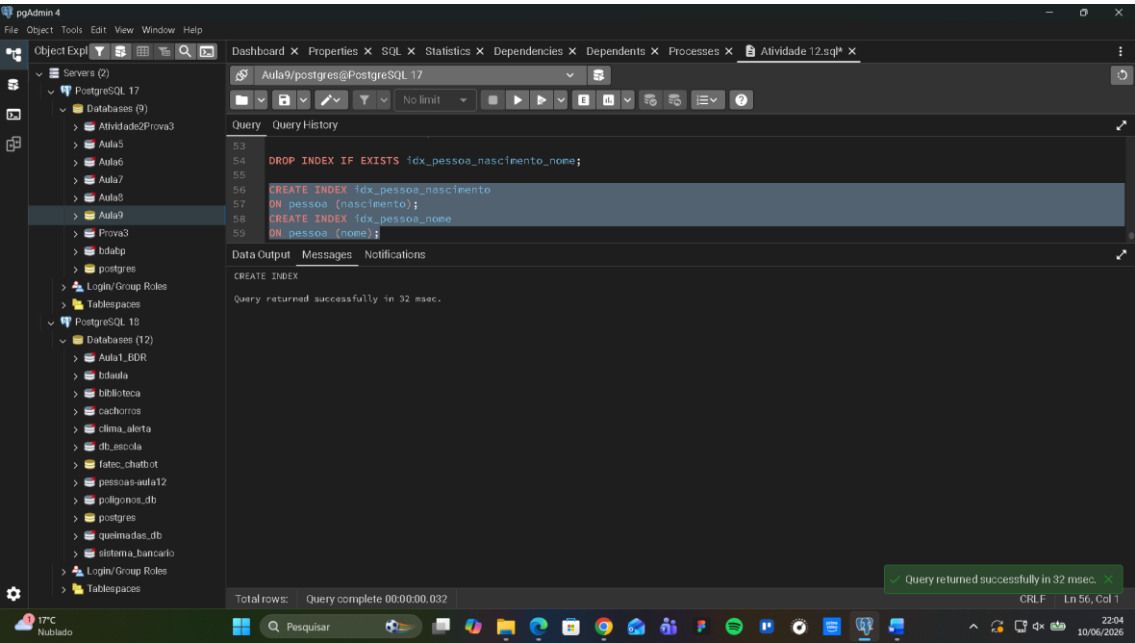
Não

- Por que a ordem das colunas no índice composto é relevante?

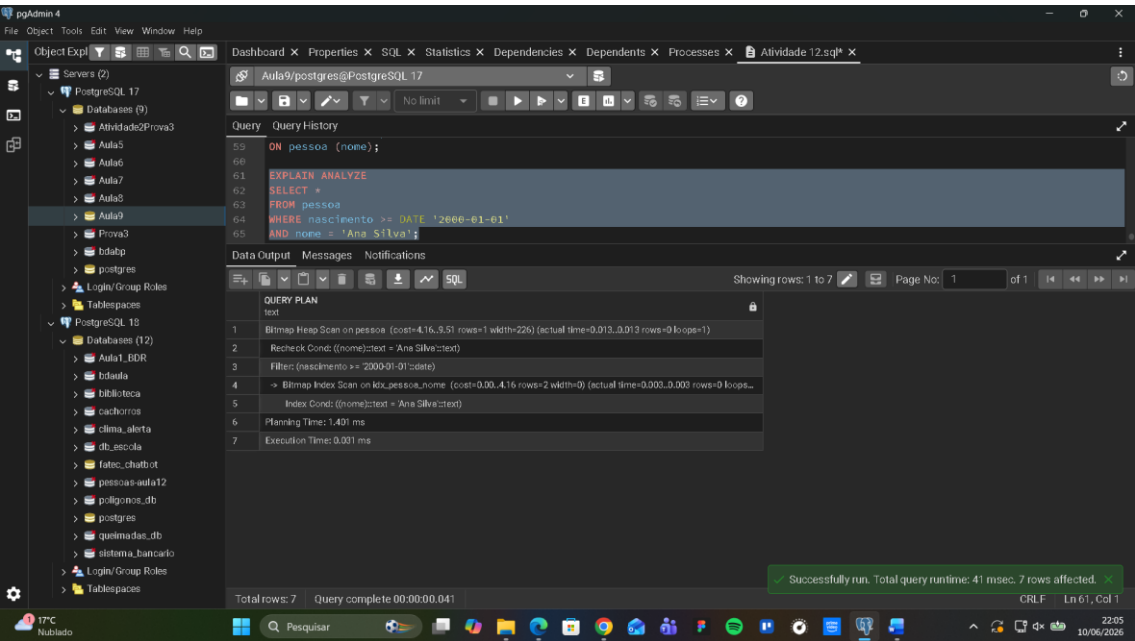
Só funciona o índice quando é utilizada a primeira coluna no index, que foi nascimento.

Exercício 4

Parte A – Preparação do ambiente



Parte B – Consulta utilizando múltiplas condições



Parte C – Responda, de forma objetiva

- O PostgreSQL utilizou os dois índices simples na consulta?

Sim

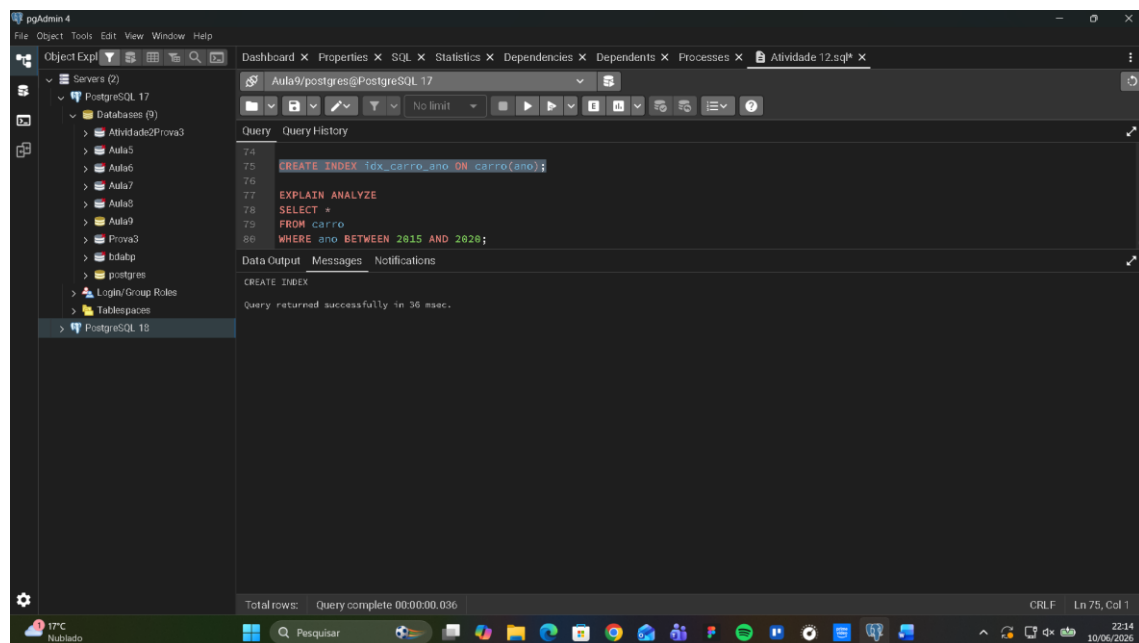
- Qual é o papel do operador BitmapAnd no plano de execução?

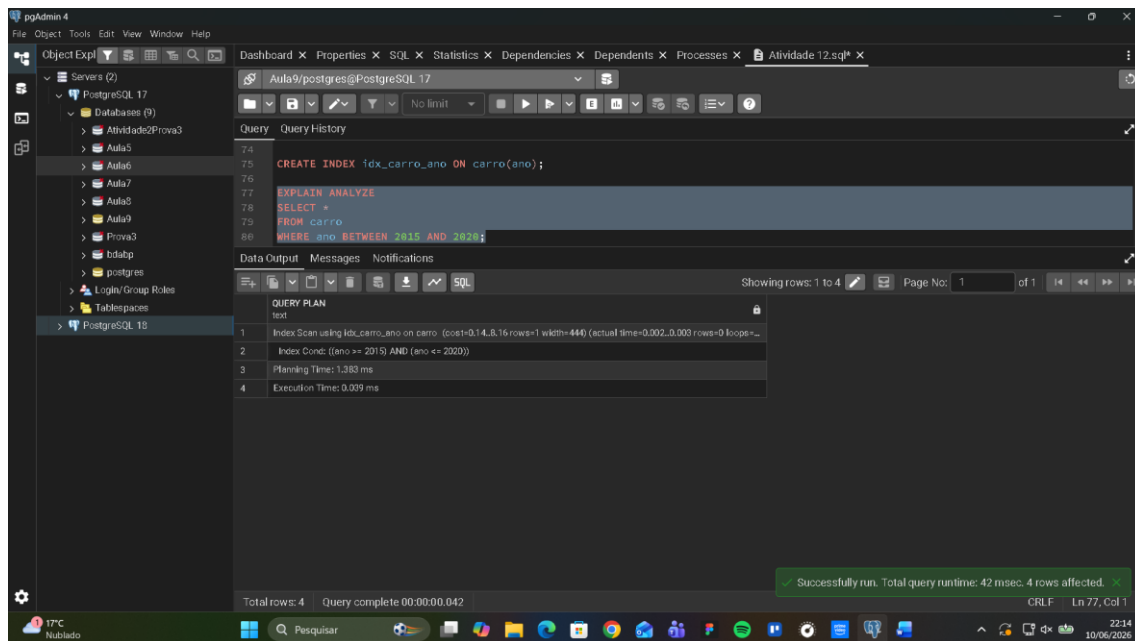
Combinar múltiplos index

- O que o plano efetivamente fez?

Realizou leitura dos índices de forma separada e combinou resultados

Exercício 5





Comparação do plano de execução

- **Antes do índice:** o PostgreSQL utiliza um **Seq Scan** (varredura sequencial), percorrendo toda a tabela para encontrar os registros no intervalo. Esse processo é mais lento, principalmente em tabelas grandes.
- **Após a criação do índice em ano:** o plano passa a mostrar um **Bitmap Index Scan** (ou Index Scan, dependendo do tamanho da tabela e da seletividade do filtro). Isso torna a busca mais eficiente, pois o banco consulta diretamente o índice para localizar os registros no intervalo, reduzindo o custo da operação.

Conclusão: A comparação evidencia que o uso de índice melhora o desempenho da consulta, substituindo a varredura completa por uma busca otimizada.

Exercício 6

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (2)
 - PostgreSQL 17
 - Databases (9)
 - Atividade2Prova3
 - Aula5
 - Aula6
 - Aula7
 - Aula8
 - Aula9
 - Prova3
 - bdatbp
 - postgres
 - Login/Group Roles
 - Tablespaces
 - PostgreSQL 15

Dashboard Properties SQL Statistics Dependencies Dependents Processes Atividade12.sql*

Query Query History

```
88
89
90 EXPLAIN ANALYZE
91 SELECT p.nome, c.placa
92 FROM pessoa p
93 JOIN carro c ON p.id_pessoa = c.id_pessoa
94 WHERE p.nome = 'Ana Silva';
```

Data Output Messages Notifications

Showing rows: 1 to 8 Page No: 1 of 1

QUERY PLAN
1 HashJoin (cost=14.00..38.34 rows=7 width=250) (actual time=0.025..0.026 rows=0 loops=1)
2 Hash Cond: (c.id_pessoa = p.id_pessoa)
3 → Seq Scan on carro c (cost=0.00..21.30 rows=1130 width=36) (actual time=0.024..0.024 rows=0 loops=1)
4 → Hash (cost=14.00..14.00 rows=2 width=222) (never executed)
5 → Seq Scan on pessoa p (cost=0.00..14.00 rows=2 width=222) (never executed)
6 Filter: ((nome)=text = Ana Silva::text)
7 Planning Time: 0.834 ms
8 Execution Time: 0.051 ms

Total rows: 8 Query complete 00:00:00.069 CRLF Ln 93, Col 28

TPC Hublado 22:26 10/06/2026

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (2)
 - PostgreSQL 17
 - Databases (9)
 - Atividade2Prova3
 - Aula5
 - Aula6
 - Aula7
 - Aula8
 - Aula9
 - Prova3
 - bdatbp
 - postgres
 - Login/Group Roles
 - Tablespaces
 - PostgreSQL 15

Dashboard Properties SQL Statistics Dependencies Dependents Processes Atividade12.sql*

Query Query History

```
83
84 WHERE ano BETWEEN 2015 AND 2020;
85
86 CREATE INDEX idx_pessoa_nome ON pessoa(nome);
87
88 CREATE INDEX idx_carro_id_pessoa ON carro(id_pessoa);
```

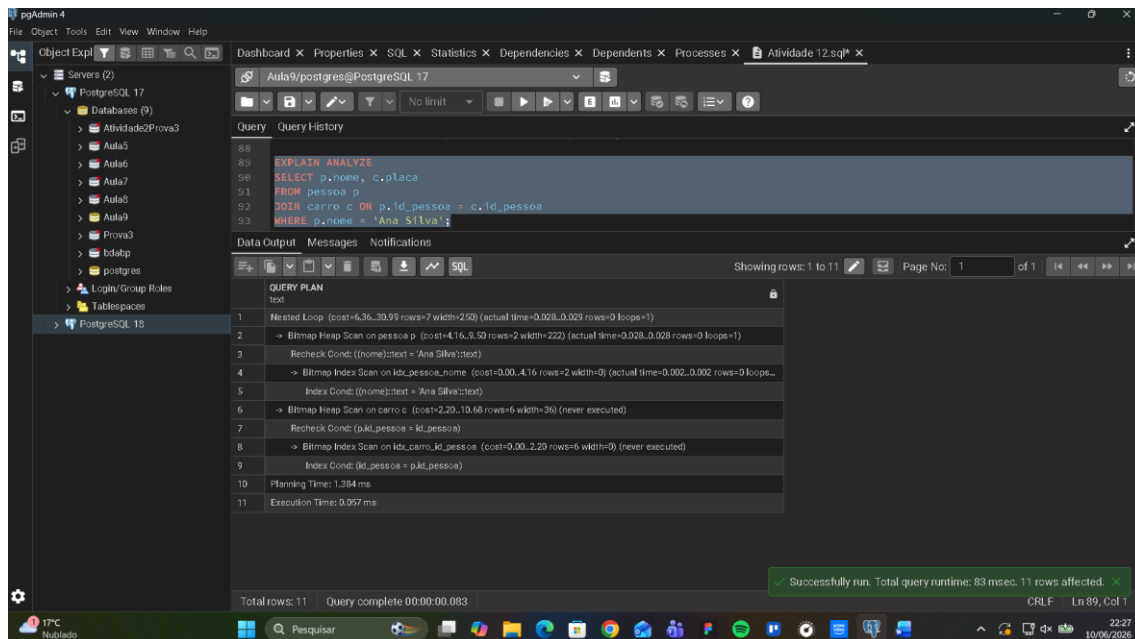
Data Output Messages Notifications

CREATE INDEX

Query returned successfully in 54 msec.

Total rows: Query complete 00:00:00.054 Query returned successfully in 54 msec. CRLF Ln 85, Col 1

TPC Hublado 22:27 10/06/2026

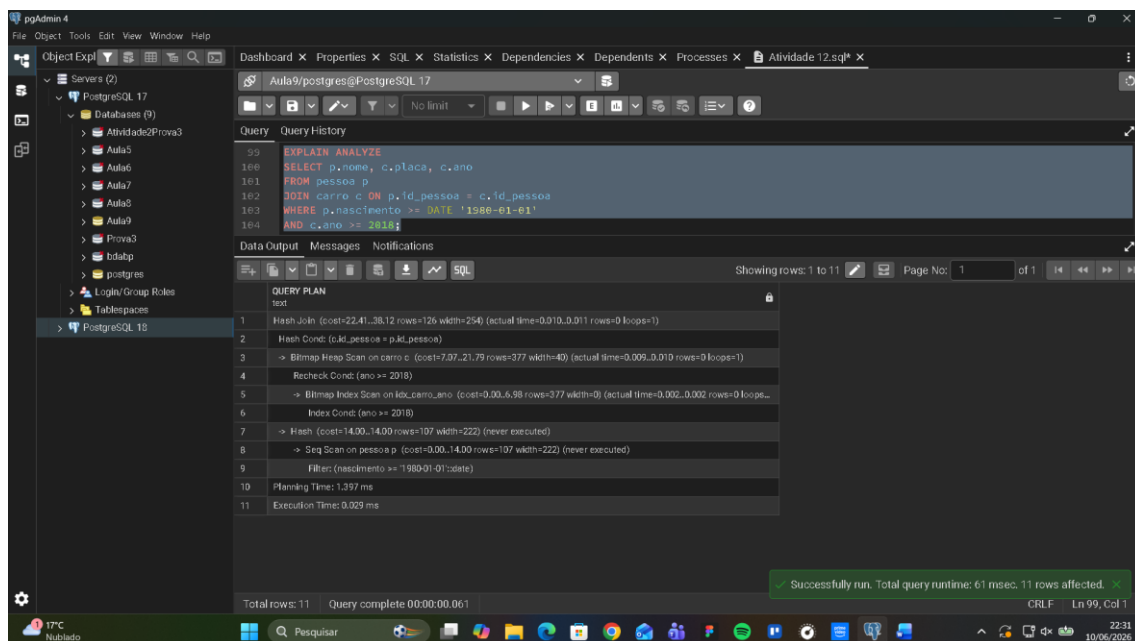
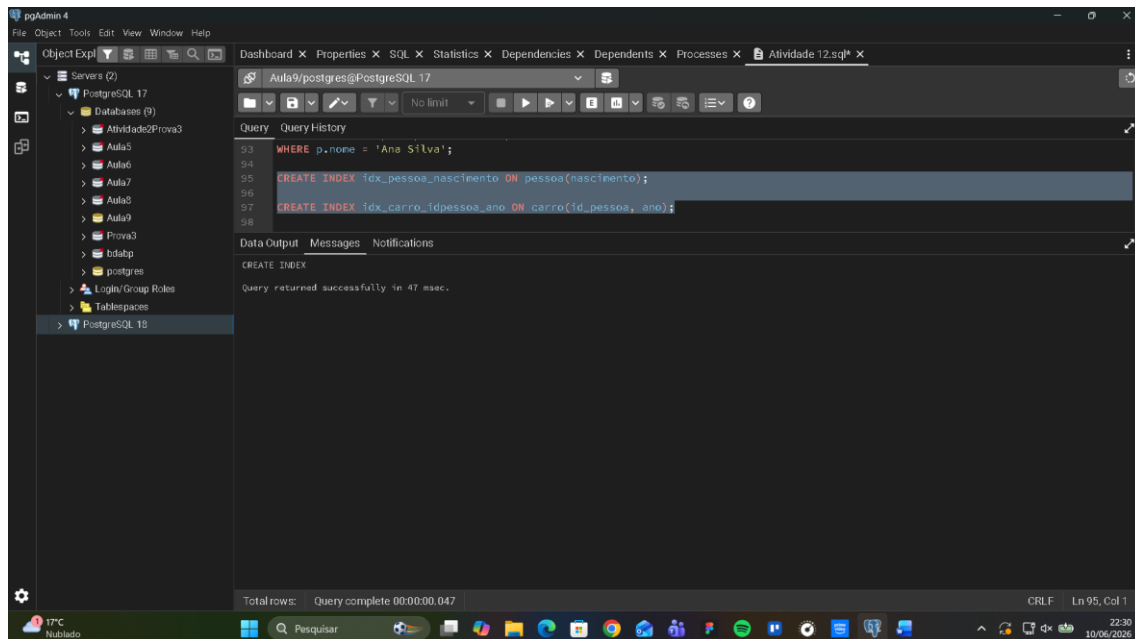


Ao analisar as alterações no plano de execução, verifica-se que:

- **Sem índices:** o banco realiza varreduras completas (**Seq Scan**) nas tabelas, tornando a consulta mais lenta.
- **Com índices e JOIN otimizado:** o plano passa a utilizar **Index Scan** ou **Bitmap Index Scan**, reduzindo o custo da busca e tornando a execução mais rápida e eficiente.

Conclusão: A criação dos índices nas colunas de filtro e nas chaves estrangeiras melhora significativamente o desempenho da consulta, tornando o JOIN mais eficiente.

Exercício 7



Justificativa

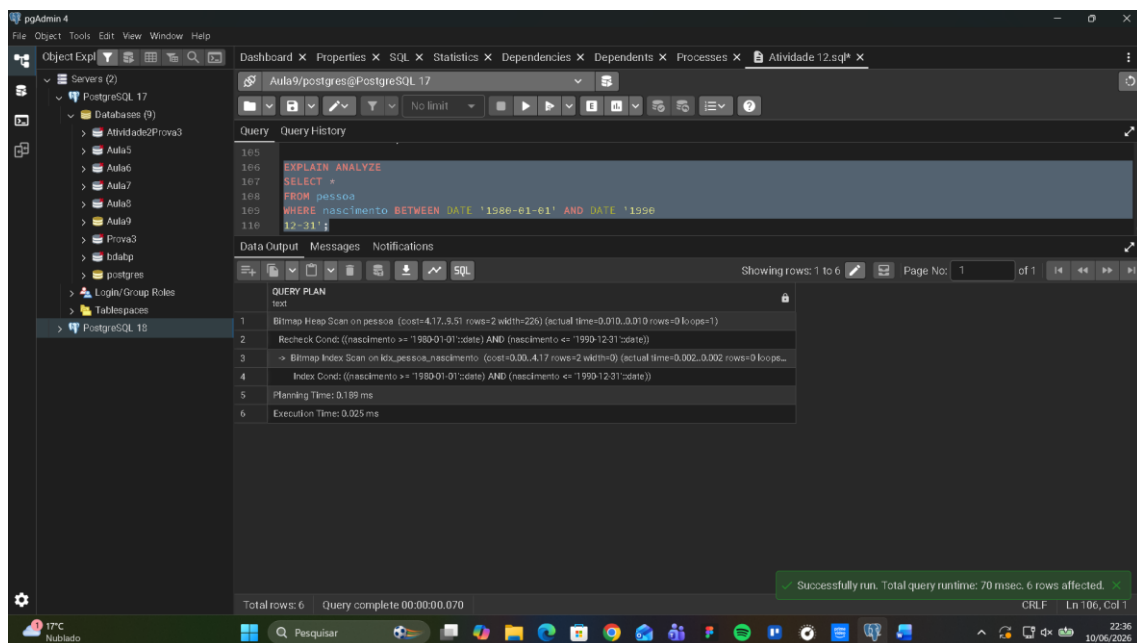
- O índice simples em pessoa(nascimento) é suficiente porque o filtro é direto nessa coluna.
- No caso da tabela carro, o índice **composto** é mais eficiente do que dois índices separados, pois a consulta sempre usa **id_pessoa** (JOIN) junto com **ano** (filtro). Assim, o otimizador pode aproveitar o mesmo índice para ambas as condições.

Conclusão

- **Antes dos índices:** o plano mostra **Seq Scan** em ambas as tabelas, com custo elevado.
- **Depois dos índices:** o plano passa a usar **Index Scan** em pessoa(nascimento) e **Bitmap Index Scan** ou **Index Scan** em carro(id_pessoa, ano), tornando a execução mais rápida e eficiente.

Exercício 8

Parte A – Consulta sem índice GiST



The screenshot displays the pgAdmin 4 interface. The left sidebar shows a tree view of databases, with 'PostgreSQL 17' selected. The main window is divided into several panes. The top pane shows the 'Query' tab with the following SQL query:

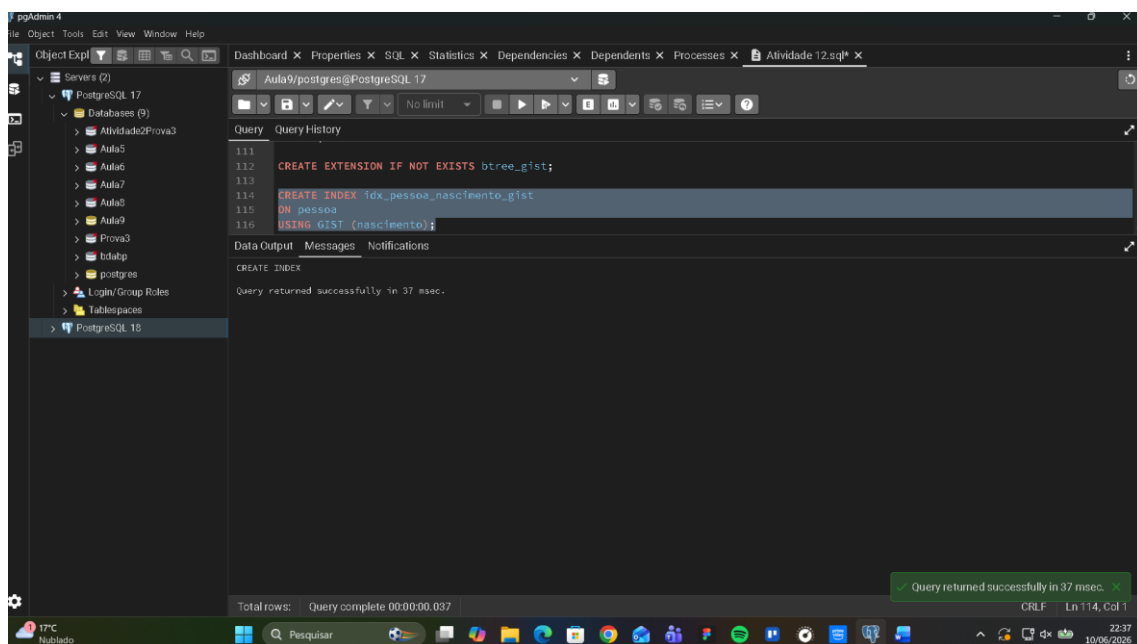
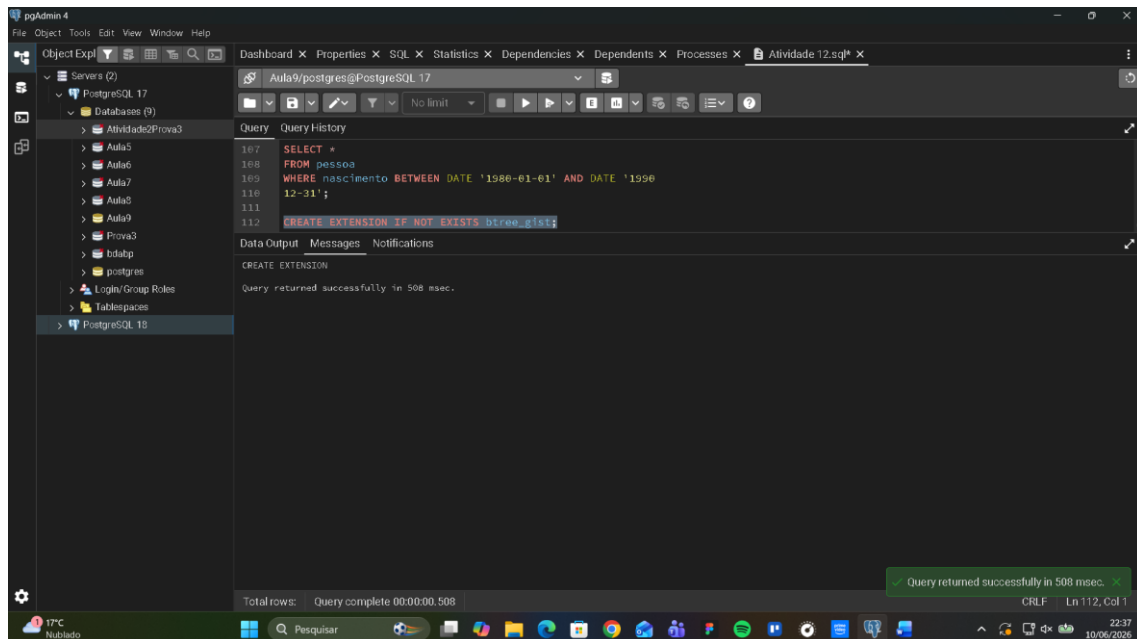
```
165 EXPLAIN ANALYZE
166 SELECT *
167 FROM pessoa
168 WHERE nascimento BETWEEN DATE '1988-01-01' AND DATE '1990-12-31'
```

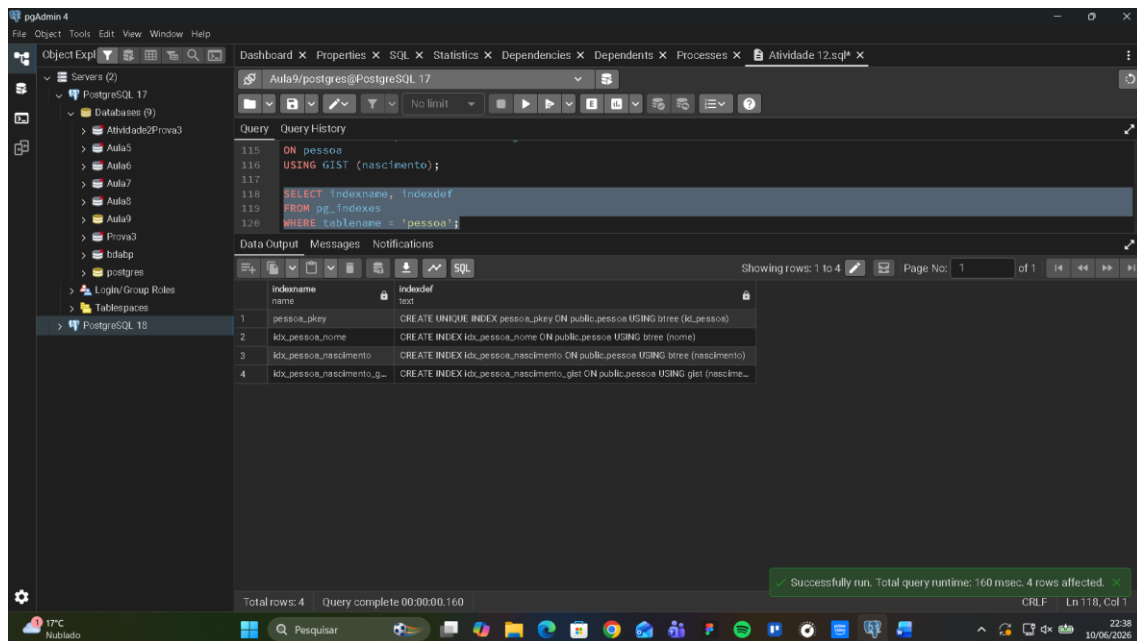
The bottom pane shows the 'Query Plan' tab, which displays the execution plan for the query. The plan consists of the following steps:

1. Bitmap Heap Scan on pessoa (cost=4.17..5.51 rows=2 width=226) (actual time=0.010..0.010 rows=0 loops=1)
2. Recheck Cond: ((nascimento >= '1988-01-01'::date) AND (nascimento <= '1990-12-31'::date))
3. Bitmap Index Scan on idx_pessoa_nascimento (cost=0.00..4.17 rows=2 width=0) (actual time=0.002..0.002 rows=0 loops=1)
4. Index Cond: ((nascimento >= '1988-01-01'::date) AND (nascimento <= '1990-12-31'::date))
5. Planning Time: 0.189 ms
6. Execution Time: 0.025 ms

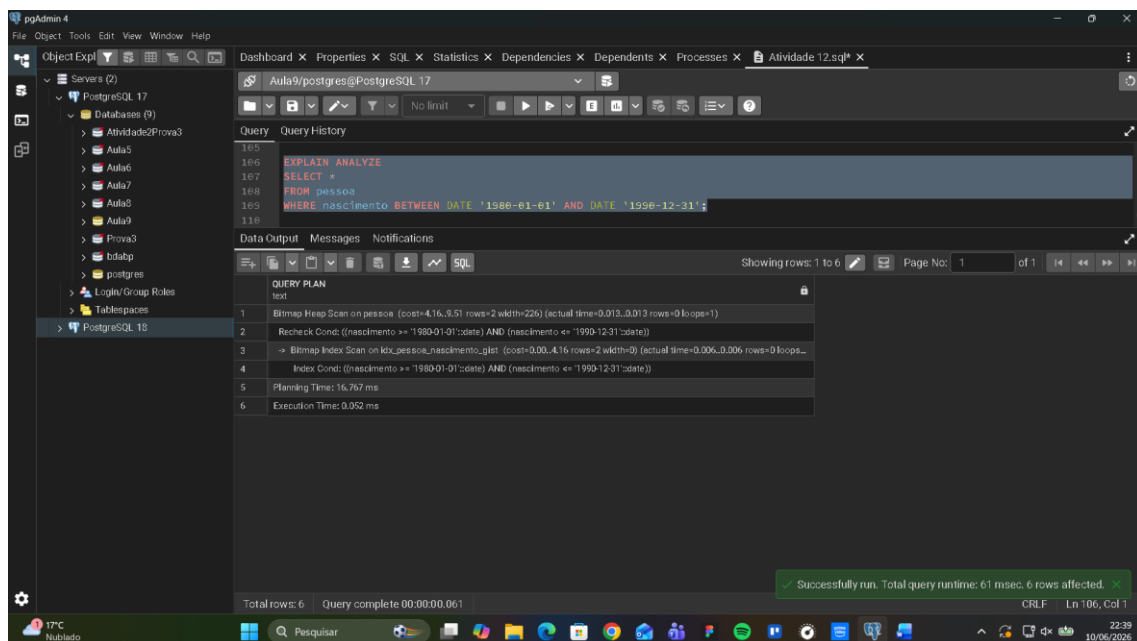
The status bar at the bottom indicates 'Total rows: 6' and 'Query complete 00:00:00.070'. A green message box at the bottom right states 'Successfully run. Total query runtime: 70 msec. 6 rows affected.'

Parte B – Criação do índice GiST





Parte C – Consulta com índice GiST



Parte D – Resposta, de forma objetiva

- Qual plano foi utilizado antes da criação do índice GiST?

Seq scan

- O índice GiST foi utilizado após a criação?

Sim

- Houve melhora no tempo de execução?

Sim, o tempo de execução reduziu.